ACTIVITY 3.3 · UNIT 3 — PROGRAMMING & AUTOMATION

Your First C++ Program

Student Edition — VEX EXP · C++ in VS Code

Name: ______________________________ **Date:** ____________ **Period:** ________

Introduction

Since week 1 you've been writing code in the starter — typing between the two purple lines and leaving everything else alone. This is where you open up those other lines and read the **whole program**: the **setup lines**, the **device list**, **int main()**, **vexcodeInit()**, and the curly braces. Once you know what each part does you're not limited to the region anymore — you can write a complete program yourself, and turn the **pseudocode** you planned in Chapter 9 into real, running code. It's the same **download-and-run** loop you already know, now with nothing hidden. Read Chapter 10 in the textbook for the full walkthrough.

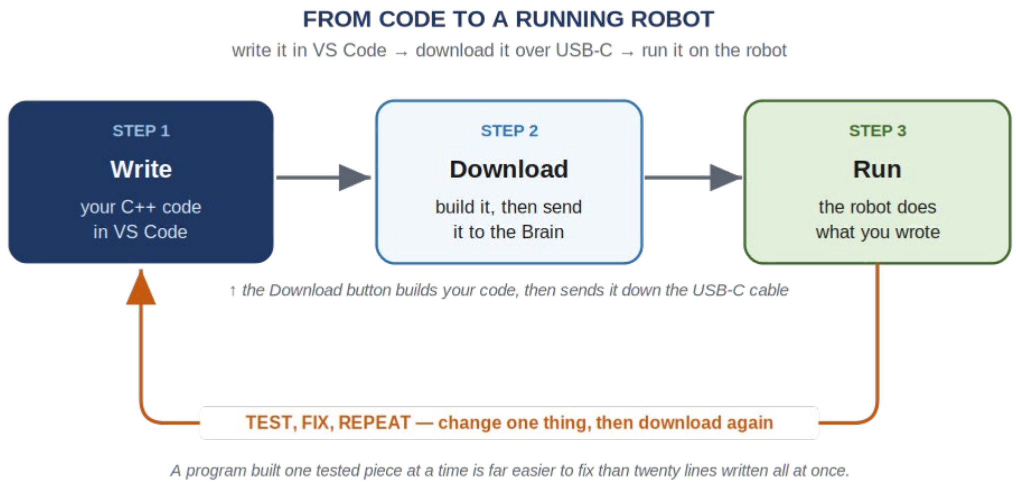
Learning Objectives

- Set up VS Code with the VEX extension and create an EXP project.
- Identify the parts of a VEX C++ program — the setup lines, the device list, int main(), and vexcodeInit() — and say what each does.
- Write motor commands (setVelocity in percent, spin, wait, stop) to make a motor run for a set time, then stop.
- Download a program to the EXP Brain over USB-C and run it, building and testing one piece at a time.
- Wire devices to the right ports on the EXP Brain (numbered Smart Ports, lettered 3-Wire ports), match the code's port to what is plugged in, power the Brain on and off, and run a first check when nothing happens.

Key Ideas (quick reference)

- **Program anatomy:** the setup lines (#include "vex.h", using namespace vex;) bring in the VEX commands; the device list names each motor and its Smart Port; int main() { } is where the robot starts and runs each line top to bottom; vexcodeInit() is the first line inside main and must stay. Every command ends with a semicolon.
- **Talking to a motor:** setVelocity sets speed as a percent (0–100; a new motor starts at 50%); spin(forward) or spin(reverse) starts it; wait(2, seconds) pauses the whole program (also msec); stop() halts it. Shortcut: spinFor(forward, 5, seconds) does start-wait-stop in one line.
- **Direction and the EXP motor:** reverse a backwards-mounted motor once where you name it — motor rightMotor = motor(PORT6, true);. Your EXP Smart Motor is fixed-speed (Chapter 3), so there is no gear cartridge to choose, unlike the V5 motor.
- **Ports & wiring:** the EXP Brain has 10 numbered Smart Ports (Smart Motor, Distance, Optical, via a Smart Cable that clicks in) and 8 lettered 3-Wire ports A-H (Bumper, Potentiometer V2); the Inertial is built in. Always name each device in code to match the port it is plugged into — a mismatch is the usual reason a new program does nothing. Power on with the Check button, off with the X button.
- **Download & run:** connect the Brain by USB-C, click Download (it builds your code, then sends it to the Brain), then Run it on the bench — or unplug and run from the Brain's screen for a robot that drives. Build a little, test a lot.

From Code to a Running Robot



Write your code in VS Code, download it to the Brain over USB-C, and run it — then test, change one thing, and download again.

The Program, Line by Line

A complete program — and you'll recognize most of it. It's the AR-Starter you've been using: the setup lines, your motor, main, and a few commands in the middle (this one sets a speed, then spins for five seconds and stops). This time we read every line, including the ones you've been skipping:

The whole program — nothing hidden this time

```
// setup — brings in all the VEX commands
#include "vex.h"
using namespace vex;

// your device list — name each motor and its Smart Port
motor leftMotor = motor(PORT1);

int main() {
    vexcodeInit();   // gets the robot ready — don't delete

    // spin at half speed for 5 seconds, then stop
    leftMotor.setVelocity(50, percent);
    leftMotor.spin(forward);
    wait(5, seconds);
    leftMotor.stop();
}
```

Every program has this skeleton (a new project writes most of it for you): the **setup lines** bring in the VEX commands; the **device list** names each motor and its Smart Port; **int main()** is where the robot starts and runs each line in order to the closing brace; **vexcodeInit()** gets the robot ready and must stay. Every command ends with a **semicolon** — "this instruction is finished." Forgetting one is the most common first mistake, and the editor underlines it in red.

Talking to a Motor

Almost everything you do to a motor is one of four commands — the motor's name, a dot, then what to do:

```
leftMotor.setVelocity(50, percent);
leftMotor.spin(forward);
wait(2, seconds);
```

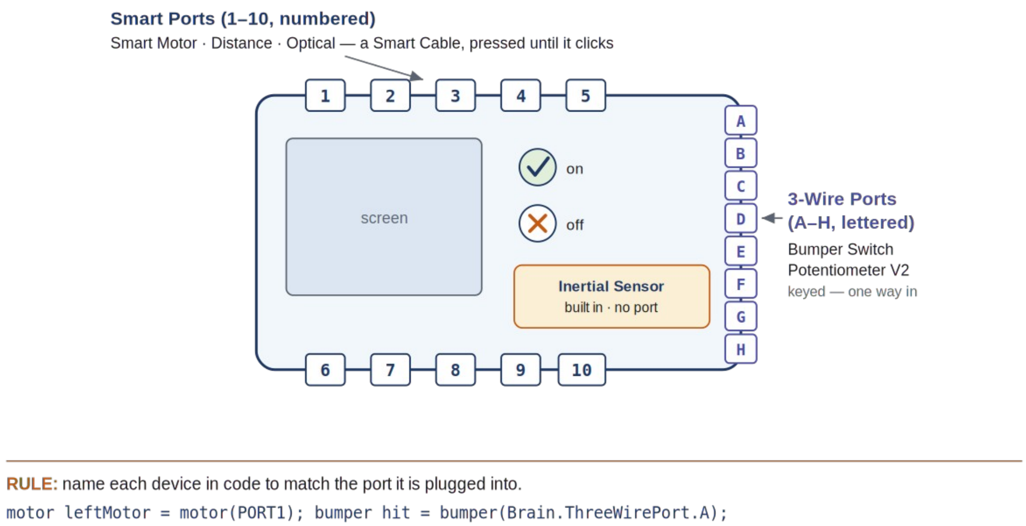
```
leftMotor.stop();
```

Speed is a **percent, 0 to 100** (50 is half, 100 is full; a new motor starts at 50%). Direction is **forward** or **reverse**; flag a backwards-mounted motor once where you name it: motor rightMotor = motor(PORT6, true);. **wait** pauses the whole program (seconds or msec) while what you started keeps going — that's why spin, then wait(5, seconds), then stop runs the motor for five seconds. Leave out **stop** and the motor keeps spinning — the classic Chapter 9 bug.

Wiring the Brain: Ports & Power

Your code names each device and the port it is plugged into (motor leftMotor = motor(PORT1);), so the motor must really be in port 1. The EXP Brain has two kinds of port, plus one sensor built in:

10 Smart Ports (numbered 1-10, top and bottom edges): the Smart Motor, Distance, and Optical sensors plug in with a Smart Cable, named by number — motor(PORT1), distance(PORT2).
8 3-Wire Ports (lettered A-H): simpler devices — the Bumper Switch and Potentiometer V2 — named by letter, bumper(Brain.ThreeWirePort.A).
Inertial Sensor — built into the Brain; there is no port, and it is always available (Chapter 13).



The EXP Brain: 10 numbered Smart Ports (Smart Motor, Distance, Optical), 8 lettered 3-Wire ports A-H (Bumper, Potentiometer V2), and a built-in Inertial. Name each device in code to match its port.

Push a Smart Cable firmly into both the device and the Brain until it clicks; remove it with the release tab, not by pulling the wire. 3-Wire connectors are keyed and go in one way. **Match the code to the ports** — the most common reason a new program does nothing is the code naming one port while the device is in another. **Power:** a charged battery slides into the back; press the Check button to turn on, the X button to turn off. (The EXP battery and charger are their own — they do not swap with the V5 team's kit.)

It downloaded but nothing moves? Check in order: (1) cable seated — re-push till it clicks; (2) right port — does the code's PORT1 match where it is plugged? The Brain's Devices screen and the VEX DEVICE INFO panel in VS Code both show what the Brain detects; (3) powered and charged; (4) named in the device list; (5) motor whining but not turning = too much load (Chapter 3's stall).

Lab Setup & Ground Rules

> **Lab setup & ground rules:** (1) You need a computer with VS Code and the VEX extension

> installed, the classroom kit's EXP Brain with a charged battery, an EXP Smart Motor (or your robot), and a USB-C cable. The work happens on the computer in front of you — not in the cloud — because your program has to travel down the cable into the Brain. (2) Don't borrow the separate competition team's V5 brain or motors; they're a different system and stay in their own pool. (3) Before you run anything, make sure the robot can't drive off the bench, and only run a driving program after you've unplugged the USB-C cable.

Safety

> • Before you run a program, make sure the robot can't drive off the bench — prop the wheels up or be ready to grab it. A first program can lurch unexpectedly.
>
> • Keep fingers, hair, and loose clothing clear of the spinning motor shaft and any gear or wheel on it.
>
> • Run a driving program only after you've unplugged the USB-C cable — never let the robot drive while tethered to the computer.
>
> • Seat the USB-C cable gently, and power the Brain down with the X button when you are done.

Equipment & Materials

- A computer with VS Code and the VEX extension installed
- A VEX EXP Brain and a charged EXP battery
- At least one EXP Smart Motor (or your robot from earlier chapters)
- A USB-C cable to connect the Brain to the computer
- Your pseudocode from Chapter 9, and your engineering notebook

Procedure

1. **Create a project.** In VS Code, make a new EXP project and name it (your name shows up on the Brain's screen). Open the file in the src folder.
2. **Name your motor.** Add the device line for your motor near the top — its name and the Smart Port it's plugged into, like motor leftMotor = motor(PORT1);.
3. **Type the first program.** Inside main, after vexcodeInit();, write the four lines that set the speed, spin forward, wait five seconds, and stop.
4. **Download and run.** Plug in the Brain over USB-C, click Download, then Run. Watch the motor spin for five seconds and stop. If the build complains, fix the first error and try again.
5. **Add a second motor (build a little, test a lot).** Name a second motor on another port, mounted on the other side so set it reversed. Make both motors spin, run a few seconds, and stop. Download and test — then try one forward and one reverse.
6. **Make your plan real.** Take one behavior from your Chapter 9 pseudocode and translate it into C++, one line at a time, downloading and testing as you go.

Worksheet 1 — Label the program

For each part of a VEX C++ program, write what it does.

Program part	What does it do?
#include "vex.h"	
motor leftMotor = motor(PORT1);	

Program part	What does it do?
int main() { ... }	
vexcodeInit();	
leftMotor.spin(forward);	
the semicolon ; at the end of a line	

Worksheet 2 — Write the code

Write the C++ command(s) for each behavior. Don't forget the **semicolons**.

Behavior (in English)	Your C++ command(s)
Set armMotor to full speed	
Spin leftMotor forward	
Wait 3 seconds	
Stop leftMotor	
Spin armMotor at full speed for 3 seconds, then stop	

Worksheet 3 — Trace the program

Read this program and describe, start to finish, **what the robot does** (assume both motors drive the wheels):

```
leftMotor.setVelocity(50, percent);
rightMotor.setVelocity(50, percent);
leftMotor.spin(forward);
rightMotor.spin(forward);
wait(3, seconds);
leftMotor.stop();
rightMotor.stop();
```

Your answer

Conclusion Questions

Answer in complete sentences.

1. Name the parts of a VEX C++ program: what do the setup lines, the device list, int main(), and vexcodeInit() each do?
2. What does a semicolon mean at the end of a C++ command, and what usually happens if you forget one?
3. Write the commands to spin a motor named armMotor at full speed for 3 seconds, then stop.
4. What does the Download button do — and why isn't writing code alone enough to move the robot?
5. You wrote spin and wait(3, seconds) but left out stop. What will the motor do, and why?
6. Why is it better to write and test one command at a time than to type the whole program first? Connect your answer to the test loop in the diagram.
7. Your code says motor(PORT1) but the motor is plugged into port 3, and nothing happens when you run it. What is wrong, and what two things on the screen could you check to see which port the Brain actually detects the motor in?